



Software Engineering Graduation Project

Course Roadmap and Project Report Structure

GRADUATION PROJECT INTRODUCTORY COURSE

Why This Course Exists?

Course Objective

This course is designed to integrate all your software engineering knowledge under a real project, develop your teamwork and project management skills, and provide an industry-standard development experience.

Deliverables at the End of the Semester

- *A working software prototype (MVP)*
- *A professional engineering report*
- *Final presentation in front of the class*
- *Source code repository*

These three outputs will prepare you for real-world projects and provide a tangible contribution to your resume.



Course Overview

Throughout the semester, you will go through eight phases that simulate a real software engineering lifecycle. Each phase builds the foundation for the next.



This workflow accurately reflects the real software engineering cycle used in the industry.

Development Methodology: Agile + Scrum

Why Scrum?

Scrum divides complex projects into manageable sprints. At the end of each sprint, a working piece of software emerges. This approach provides both flexibility and measurable progress.

Sprint Planning

Task distribution

Development

Coding & integration

Testing

Validation & debugging

Sprint Review

Feedback & improvement

Goal: MVP

With each sprint, you gradually build the core of your system, reaching a **Minimum Viable Product (MVP)**. An MVP is the first working product that offers essential functionality and can be experienced by real users.



Final Report Structure

Your final report will consist of the following 12 sections. Each section aligns with a phase of the course, and each section will be explained in detail in the following slides.

01

Introduction & Background

Problem definition, motivation, and related concepts

02

Literature Review & Process Model

Analysis of existing studies and Scrum methodology

03

Requirements & Modeling

Functional requirements and system diagrams

04

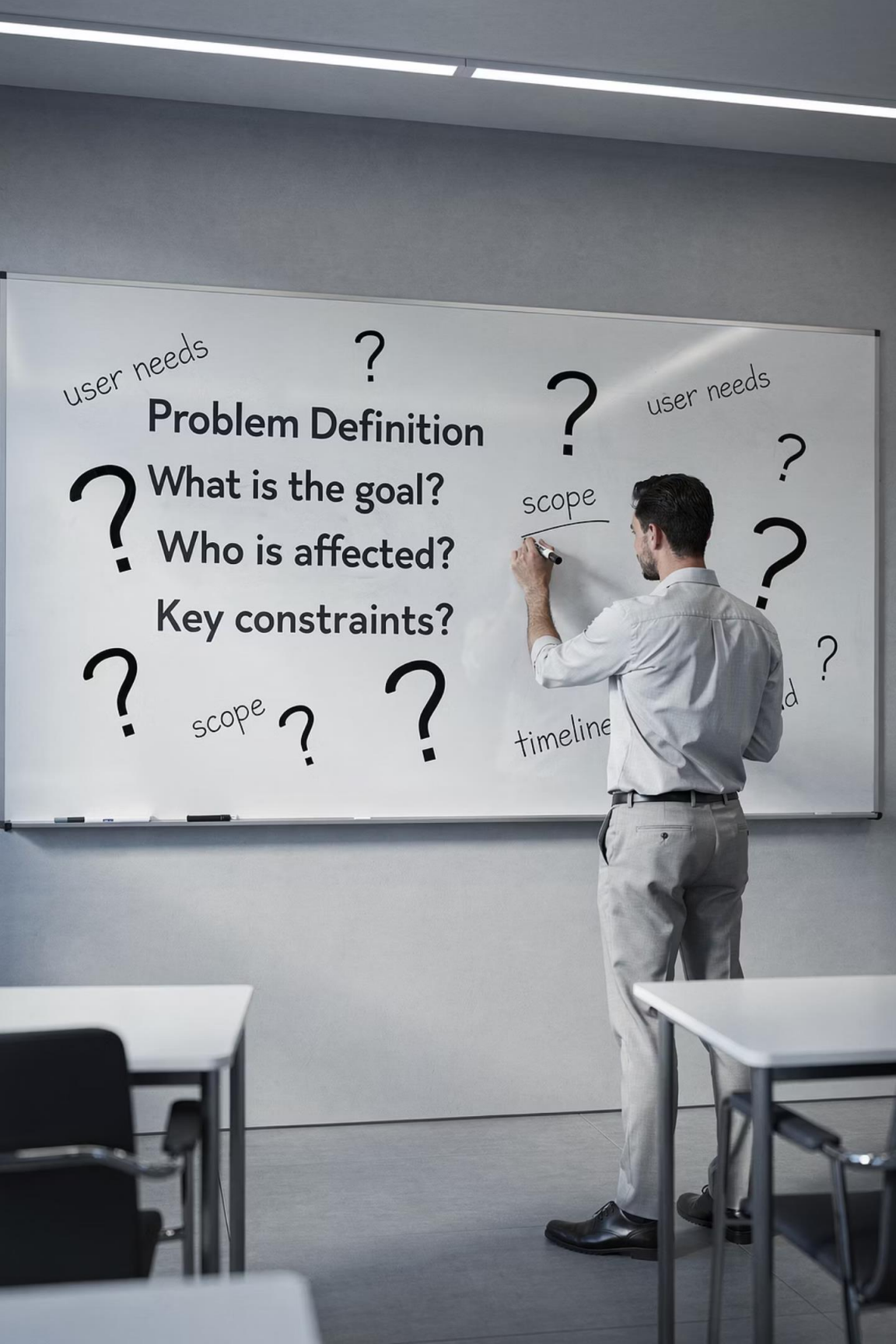
Architecture & Implementation

System design, MVP development, and testing

05

Results & Future Work

Evaluation, discussion, and recommendations



SECTION 1

Introduction

What to Write in This Section?

- *Clearly define the problem your project solves*
- *Justify why this problem is important*
- *Specify the target audience and stakeholders*
- *Outline the system's objectives and scope*

Key Question

What problem are we solving, and why is this problem worth solving?

A good introduction convinces the reader of the project's motivation and prepares a strong foundation for the rest of the report. Avoid ambiguity — every sentence should be purposeful.

Background and Core Concepts

< p>In this section, you introduce the application area of your project and the core technologies it relies on. You should provide the necessary information for a reader unfamiliar with the technical background to understand the project.



Artificial Intelligence & ML

Explanation of fundamental AI concepts such as machine learning, deep learning, and NLP within the project context



Cloud Architecture

Technical explanation of the cloud services and server infrastructure you use



Web Architectures

Justification of architectural decisions such as REST API, microservices, or monolithic structure



Application Domain

Basic information and terminology about the industry or field in which your project operates

Literature Review

What Should Be Done?

- *Examine similar applications and competing systems*
- *Review academic articles and technical reports on the subject*
- *Compare the strengths and weaknesses of existing solutions*

Goal

*A literature review is how you prove **how your project differentiates itself** from or **improves upon** existing work. Without this section, you cannot demonstrate your project's unique contribution.*

*It is recommended to review at least **5–10 sources** and present them in a comparative table.*



Software Process Model

*In this section, you explain **how you developed** your project from a methodological perspective. You should justify not only what you did, but also why you chose this method.*

1

Why Scrum?

Adaptation to changing requirements, short feedback loops, and continuous delivery

2

Team Roles

Distribution of Product Owner, Scrum Master, and developer roles within the team

3

Sprint Organization

Planning of sprint duration, backlog management, and iteration cycles

4

Workflow

Arrangement of daily stand-ups, sprint reviews, and retrospective meetings

Requirements Analysis

*This is one of the most critical sections of the report. It clearly defines **what is expected** from the system. Ambiguous requirements lead to serious problems in later stages of the project.*

Functional Requirements

*Defines **what the system should do**.*

- *Users should be able to create accounts*
- *Users should be able to upload data*
- *The system should visualize results*
- *Administrators should be able to manage users*

Non-Functional Requirements

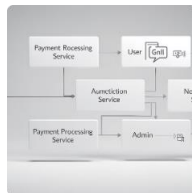
*Defines **how the system should behave**.*

- ***Performance:** Page load time < 2 sec*
- ***Security:** Encrypted authentication*
- ***Scalability:** 1000+ concurrent users*
- ***Availability:** 99.9% uptime target*

SECTION 6

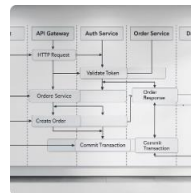
System Modeling

*This section defines the system through **visual diagrams**. Well-drawn diagrams convey much more than written descriptions and facilitate communication with stakeholders.*



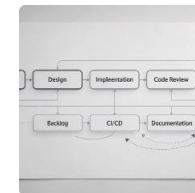
Use Case Diagram

*Shows user-system interactions and actors.
Reveals how the system looks from the outside.*



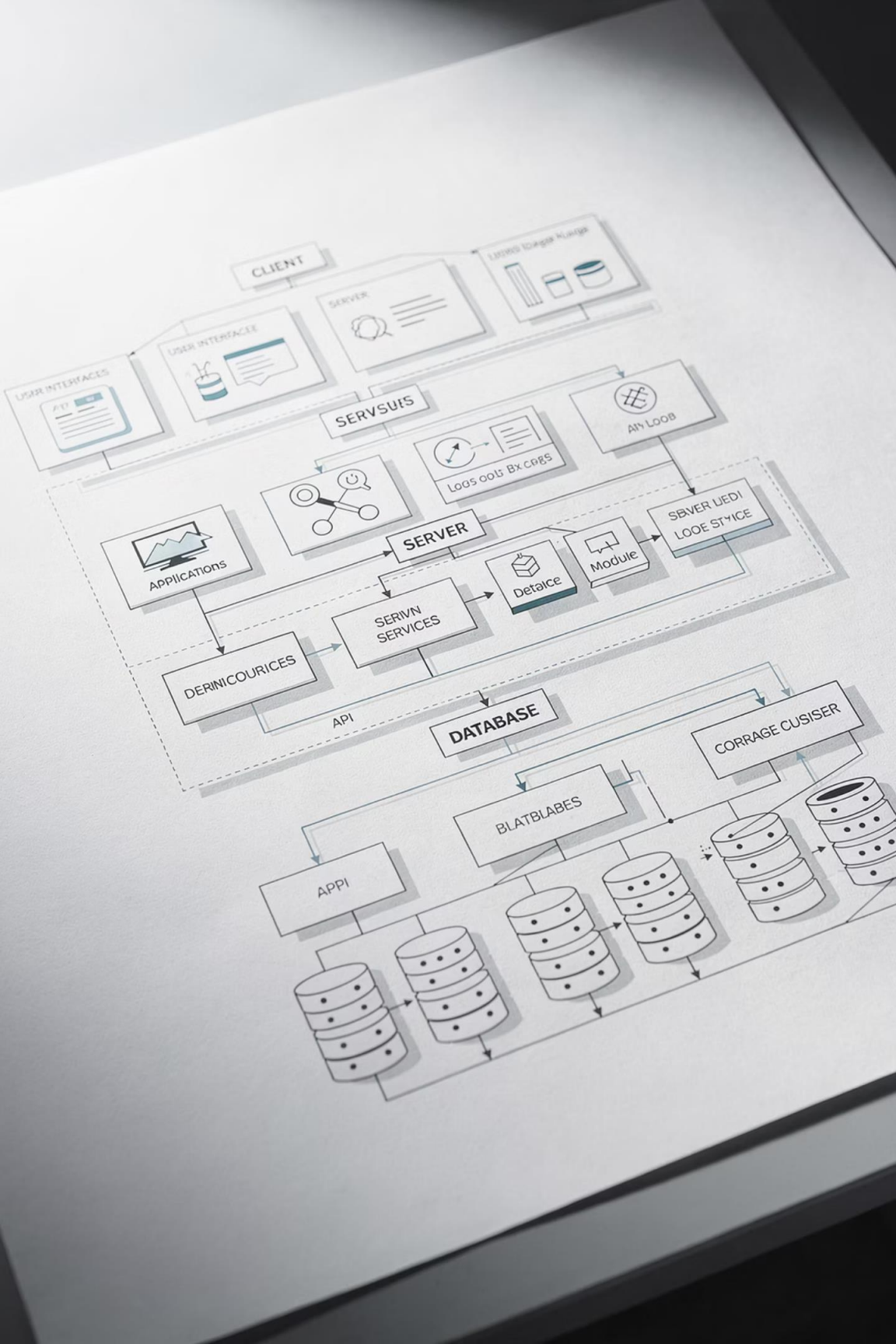
Sequence Diagram

Shows the message flow and time order between components. Explains complex processes step-by-step.



Activity & Context Diagrams

These are diagrams that model business processes and the system's relationship with its external environment.



SECTION 7

System Architecture and Design

What's in This Section?

- *System architecture and component diagram*
- *Database design (ER diagram)*
- *Class diagrams and object model*
- *Interface design (wireframe)*

Possible Architectures

Client-Server

Classic, reliable structure

Microservices

Scalable, independent modules

Monolithic

Simple, fast prototyping

Implementation

*In this section, you describe the technical details of **how you built** the system. Justify the technology stack you used and explain your code organization.*



Programming Languages

Python, Java, JavaScript, Swift, or choice and justification of language suitable for your project



Frameworks & Tools

Used frameworks and libraries such as React, Node.js, Django, Spring Boot



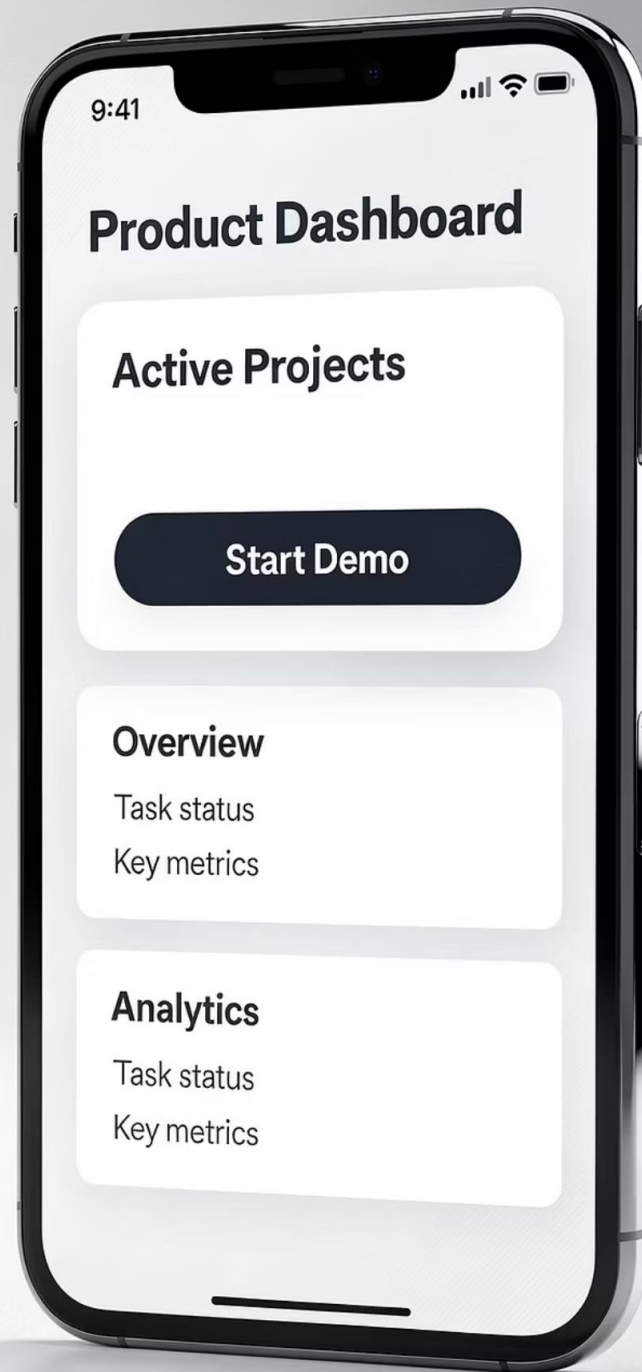
System Modules

Main components of the project, module structure, and distribution of responsibilities



Code Structure & Version Control

Explanation of folder structure, branching strategy, and Git workflow



SECTION 9

MVP Development

What is MVP?

Minimum Viable Product; the smallest working product that allows users to experience core value. Not fully featured — but real and functional.

- *Core system functionality*
- *A working prototype*
- *Basic user interface*

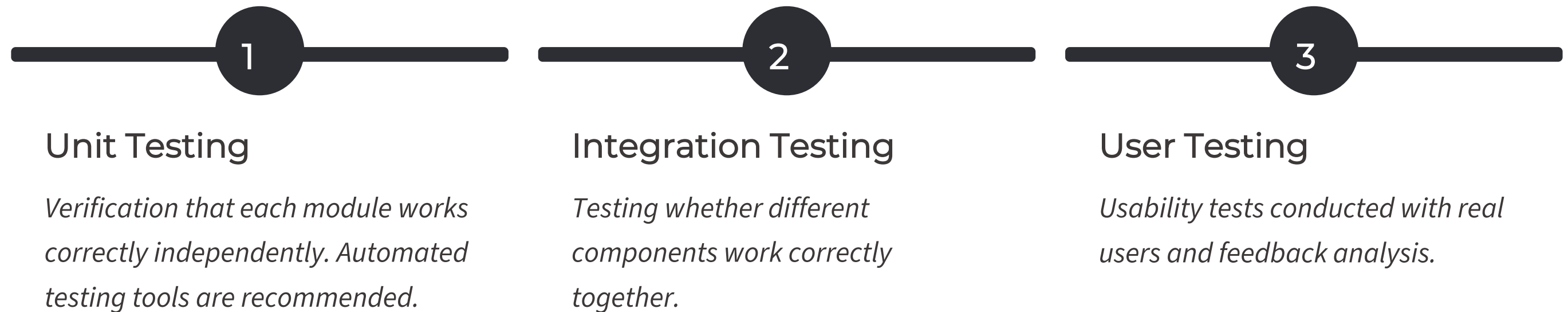
What should be presented in the report?

- *Screenshots and descriptions*
- *List of implemented features*
- *Step-by-step demonstration of system workflow*
- *Demo video or live demo link*

📄 *The MVP is central to your final presentation at the end of the term. Start early!*

Testing and Evaluation

The testing phase proves the system's reliability and compliance with requirements. Well-documented tests significantly enhance the quality of your report.



*For each test scenario, a table including **input data**, **expected outcome**, and **actual outcome** should be prepared.*

Conclusions, Discussion, and Future Work

Results & Discussion

- *Evaluate system performance based on measurements*
- *Highlight the strengths of the solution*
- *Honestly convey existing limitations*
- *Discuss whether project goals have been met*

Conclusion & Future Work

- *Summarize the project concisely*
- *Reflect on what you learned during the period*
- *Suggest features that can be added in the future*
- *Discuss ways to improve, such as scalability and AI integration*



Team Responsibilities

Task Organization

All tasks should be managed transparently via the sprint backlog. Who will do what must be clarified at the beginning of each sprint.

Development Contribution

Every member must actively contribute to coding and system development. Passive contribution is not acceptable.

Documentation

The development process must be regularly documented. Sprint notes, meeting minutes, and decisions should be recorded.

Testing & Quality

Tests should not be left until the last week. Basic tests should be performed at the end of each sprint.

End-of-Term Deliverables

At the end of the term, each team is responsible for delivering the following four core outputs in full.

1

Final Report

Technical report with all sections fully prepared

2

Project Presentation

Final presentation to be given in front of the class; a working MVP and project poster must be ready for the presentation

3

Project Poster

Academic poster summarizing the project; to be displayed during the presentation

4

Source Code Repository

Project repository on Git with a clean commit history



A working MVP demo and a printed/digital poster must be ready when coming to the presentation. Late submissions will be considered in the evaluation.

Final Recommendations

Start with a Clear Problem Definition

Vague problems lead to vague solutions. Clarify what your project aims to solve from the first week.

Define a Realistic Project Scope

Instead of trying to do everything, do the chosen features very well. Think MVP-focused.

Start Early, Iterate Frequently

Scrum's strength is iteration. Instead of leaving things for the last week, make concrete progress every sprint.

Good Documentation Makes a Difference

Reports are as important as code. What you write determines both your grade and your professional reputation.



Are you ready?

Throughout this semester, you will think, design, and produce like a real software engineer. The process will be challenging — but by the end of the semester, you will have a tangible, working product that is truly yours.



Goal

A working MVP and a professional engineering report



Method

Disciplined, iterative development with Agile + Scrum



Spirit

Teamwork, open communication, and continuous learning

Don't wait to be perfect to start. You become perfect when you start.